# Critical Analysis: Least-to-Most Prompting Enables Complex Reasoning in Large Language Models

**Kai-Yu Lu**

## 1 Methodological Strengths

### 1.1 Clear problem formulation and focus on easy-to-hard generalization

The paper formulates a precise methodological target: improving *easy-to-hard generalization* in large language models by changing the prompting protocol rather than the model architecture. The study explicitly focuses on settings where test instances are strictly harder than the few-shot exemplars, such as longer sequences in symbolic manipulation and SCAN length splits, or multi-step math problems requiring more reasoning steps than the prompt exemplars. This explicit focus on difficulty mismatch between exemplars and test items provides a clear axis along which methodological choices and empirical claims are aligned.

### 1.2 Simple and well-defined prompting protocol

The least-to-most prompting strategy is methodologically attractive because it is conceptually simple, decomposed into two clearly defined stages:

1. A *decomposition stage* in which a complex problem is mapped to a sequence of simpler subproblems via a few-shot prompt containing constant decomposition exemplars.
2. A *subproblem-solving stage* in which subproblems are answered sequentially, and the prompt explicitly includes previously solved subproblems and their answers as part of the context for subsequent subproblems.

This separation between decomposition and solving is consistently instantiated across tasks. For last-letter-concatenation and SCAN, the two stages are implemented as separate prompts, while for GSM8K a single-pass variant merges them. The design emphasises *dependent* subproblems whose solutions are reused in later steps, which encodes a recursive or dynamic programming pattern within the prompt itself.

### 1.3 Strong comparative baselines and controlled prompt design

The methodology includes carefully controlled comparisons between three prompting paradigms:

- standard few-shot prompting without rationales,
- chain-of-thought prompting with natural language reasoning,
- least-to-most prompting with explicit decomposition and reuse.

For the last-letter-concatenation task, the chain-of-thought baseline uses the *same* word lists as least-to-most prompting, with the only difference being that the reasoning for longer lists is written from scratch rather than reusing previous outputs. This design isolates the effect of reuse and decomposition from confounding factors such as vocabulary or list composition. In addition, the paper explores multiple chain-of-thought prompts with different numbers of exemplars (2, 4, 8) and different degrees of independence between examples, and reports that least-to-most prompting maintains a strong advantage even when chain-of-thought prompts are lengthened and strengthened.

For SCAN, baselines are derived by stripping intermediate explanations from the command-mapping prompt to obtain standard few-shot prompting, and by omitting the decomposition stage for chain-of-thought prompting. This construction ensures that the underlying mapping examples and intermediate representations are shared between methods, so that differences in performance can reasonably be attributed to the presence or absence of decomposition.

### 1.4 Use of diverse benchmarks targeting different reasoning skills

The experimental design covers three distinct families of tasks:

- a synthetic symbolic manipulation task (last-letter-concatenation),
- compositional command-to-action mapping (SCAN, especially the length split),
- math and discrete reasoning benchmarks (GSM8K and the numeric subset of DROP).

Each family stresses a different aspect of reasoning: length generalization in symbolic string transformations, compositional generalization in grounded action semantics, and multi-step quantitative reasoning in textual contexts. By demonstrating consistent gains for least-to-most prompting across these domains, the methodology supports the claim that the proposed strategy is not tied to one particular dataset or task formulation.

### 1.5 Explicit treatment of intermediate representations and context limits

The SCAN experiments use Python-style expressions as compact intermediate representations of action sequences in order to remain within context-length constraints of GPT-3 models. The paper validates this choice by constructing an auxiliary experiment in which the LLM is asked to expand Python expressions such as `LOOK * 2` into explicit token sequences and achieves approximately 99.7 percent accuracy. This auxiliary experiment provides empirical support that the intermediate representation itself is not a primary bottleneck, and that the main difficulty lies in learning a generalizable mapping from commands to these intermediate expressions.

### 1.6 Systematic ablations and error analysis

The methodology includes several ablations and qualitative analyses that strengthen the empirical conclusions:

- Varying the number of exemplars for chain-of-thought prompting and least-to-most prompting shows that additional exemplars improve both, but least-to-most prompting remains substantially better, especially at longer sequence lengths.
- Evaluating multiple GPT-3 variants (`code-davinci-002`, `text-davinci-002`, `code-davinci-001`) demonstrates that the superiority of least-to-most prompting holds across base models, and that code-oriented models are particularly strong for iterative and recursive reasoning.

- Error analysis for last-letter-concatenation identifies concatenation errors (adding, dropping, or permuting letters) as the dominant failure mode, whereas incorrect identification of last letters is rare. This indicates that the model has learned the subtask but sometimes fails in the final composition step.
- Error analysis for SCAN reveals systematic misinterpretations of modifiers such as "twice" and "thrice" after "around", and confusions between "after" and "and" in action sequencing, which clarifies the semantic boundaries of the method.
- For GSM8K, performance is broken down by the number of reasoning steps, showing that least-to-most prompting provides its largest gains on problems that require at least five steps.

### 1.7 Transparency and reproducibility

The paper provides the full prompt contexts for decomposition and solving in an extensive appendix, including prompts for last-letter-concatenation, SCAN, DROP, and GSM8K. Data generation procedures for the synthetic tasks are described, including vocabulary selection and sampling strategies for list lengths. These details make the experiments relatively straightforward to reproduce and facilitate further analysis or extension by other researchers.

## 2 Key Limitations

### 2.1 Limited domain coverage and benchmark scope

The evaluation focuses on a narrow set of benchmarks that, while widely used, constitute relatively stylised tasks:

- Last-letter-concatenation is entirely synthetic and tests a specific form of symbol manipulation, with little direct connection to real-world language understanding.
- SCAN commands are synthetic and based on a small symbolic grammar. Strong performance on SCAN may not automatically translate to realistic instruction-following tasks in natural language.
- For DROP, only the numeric subset is considered, and questions that require non-numeric answers are excluded. This reduces the scope of the benchmark to a special case of discrete reasoning.
- GSM8K provides realistic math word problems but covers only grade-school style arithmetic; algebraic and higher-level mathematical reasoning are not investigated.

As a result, the evidence that least-to-most prompting improves reasoning is strongest for highly structured, synthetic, or numerically focused tasks, and less informative about complex semantic or commonsense reasoning in open-domain settings.

### 2.2 Absence of computational cost and scaling analysis

The study does not provide a systematic analysis of computational cost associated with least-to-most prompting. The two-stage protocol typically requires:

- at least one forward pass for decomposition,
- a separate forward pass for each subproblem in the sequence,

- prompts that grow as subproblem answers accumulate.

This induces overhead in both token consumption and wall-clock latency compared with single-pass prompting methods. The paper does not quantify the average number of subproblems per instance, the increase in prompt length during solving, or the variability of model call counts across tasks. There is also no study of how performance or cost scales with problem size beyond the specific sequence lengths and step counts used in the experiments.

### 2.3 Dependence on manual prompt engineering

The success of least-to-most prompting is tightly coupled to the design of decomposition and solution prompts. For each domain, the authors manually construct:

- exemplars that demonstrate desired decomposition patterns (such as generating all prefixes of a list or segmenting SCAN commands into short phrases),
- solution prompts that encode specific reusable templates (such as a base case and recursive step for concatenation).

The paper acknowledges that decomposition prompts do not generalize across domains and that effective prompts must be re-designed for new tasks. However, there is no systematic procedure for prompt discovery, no formal guarantee on generalization of decomposition schemas within a domain, and no ablation that explores alternative decompositions for the same task. This reliance on hand-crafted prompts introduces a significant engineering burden and may hide implicit task-specific knowledge in the prompt text.

### 2.4 Indirect comparison with trained neural-symbolic systems

The comparison with neural-symbolic models on SCAN is based on data volume and overall accuracy, rather than on a controlled experimental setting. Neural-symbolic architectures are trained end-to-end on the SCAN training set, whereas GPT-3 is a large pretrained model that has already absorbed extensive external knowledge and is evaluated only via prompting. The paper highlights that least-to-most prompting achieves at least 99 percent accuracy on any SCAN split using only 14 exemplars, while neural-symbolic systems require over 15,000 training examples. However, there is no discussion of the relative pretraining budgets, nor of the possibility that SCAN-like patterns may be present in the GPT-3 pretraining data. This limits the strength of claims that least-to-most prompting replaces or dominates specialised neural-symbolic designs.

### 2.5 Limited exploration of failure modes in math reasoning

Although the study reports that many GSM8K problems unsolved by least-to-most prompting can be solved once a human provides a correct decomposition, the analysis of failure modes remains coarse. There is no quantitative breakdown of:

- how often failures are due to incorrect decompositions versus arithmetic mistakes,
- how decomposition errors correlate with problem structure (for example, problems with nested conditions or multiple entity types),
- how sensitive the method is to small changes in decomposition phrasing.

Similarly, for DROP, the improvement obtained from least-to-most prompting is substantial, but the paper does not present a detailed categorisation of question types where decomposition helps versus those where it does not, beyond several illustrative examples in the appendix.

### 2.6 Lack of robustness and generalization tests beyond accuracy

The evaluation metric throughout is exact-match accuracy on benchmark test sets. There is no investigation of robustness metrics such as:

- sensitivity to prompt perturbations,
- variance under different random seeds or decoding regimes,
- stability under paraphrased questions or reordered exemplars.

Moreover, because least-to-most prompting typically involves multiple queries, each of which may be stochastic under sampling-based decoding, there is no analysis of error compounding along the sequence of subproblems or of methods to mitigate such compounding beyond using deterministic decoding in the reported experiments.

## 3 Technical Bottlenecks

### 3.1 Decomposition quality as the primary bottleneck

The empirical results and the authors' own discussion suggest that decomposition quality is the central technical bottleneck. For GSM8K, nearly all unsolved problems can be addressed once a correct decomposition is supplied. For DROP, least-to-most prompting outperforms chain-of-thought prompting particularly when problems admit relatively straightforward decompositions into independent or sequential sub-questions. This indicates that:

- If the decomposition is wrong or incomplete, all subsequent subproblem solutions are compromised.
- Even with a strong base model, the protocol does not repair incorrect decomposition decisions during the solving stage.

In other words, the pipeline has a hard information bottleneck at the decomposition stage: a poor decomposition constrains the search space of possible reasoning chains regardless of model capacity.

### 3.2 Prompt length and context window limitations

Least-to-most prompting accumulates intermediate subproblem-answer pairs in the context. For tasks with long chains of reasoning or large action sequences, this can lead to:

- prompts that approach or exceed the context window,
- a growing ratio of intermediate noise to relevant information,
- increased risk that the model relies on recent artefacts or spurious patterns.

The paper partially addresses token limits in SCAN by using Python-style compressed expressions, but does not explore scenarios where even such compression is insufficient. As model context windows expand, this bottleneck may relax, but the dependence on ever longer prompts remains a structural limitation of the approach.

### 3.3 Sequential dependence and error propagation

The sequential structure of least-to-most prompting introduces a natural error-propagation pathway. Each subproblem is solved conditional on all previous answers; an early error can mislead subsequent steps in at least two ways:

- directly, when later reasoning explicitly uses incorrect intermediate values,
- indirectly, when the model infers patterns from flawed examples in the prompt history.

The paper provides qualitative examples of such compounding effects in SCAN and last-letter-concatenation, where incorrect intermediate strings or misinterpreted modifiers propagate into increasingly severe downstream errors. There is no mechanism for revising upstream answers, no explicit consistency checks, and no use of alternative reasoning paths, although the authors note that least-to-most prompting could in principle be combined with self-consistency sampling.

### 3.4 Model and representation dependence

The experiments show that least-to-most prompting strongly benefits from model choice (`code-davinci-002` versus `text-davinci-002` or `code-davinci-001`) and from specific intermediate representations (for example, Python expressions for SCAN). This reveals two technical dependencies:

- The approach assumes base models with sufficient capability to follow relatively intricate prompt instructions and to reproduce template-like reasoning patterns. Weaker models such as `code-davinci-001` perform poorly even under least-to-most prompting.
- The choice of intermediate representation is non-trivial; certain notations may align better with the model's pretraining distribution, as suggested by the strong performance of Python-style expressions in code-oriented models.

These dependencies suggest that least-to-most prompting is not a model-agnostic procedure and may need to be tuned to each base model and domain-specific representation.

## 4 Research Implications

### 4.1 Implications for compositional generalization and algorithmic reasoning

The finding that least-to-most prompting achieves at least 99 percent accuracy on SCAN length splits with only 14 exemplars has significant implications for compositional generalization research. It shows that:

- Large pretrained language models can emulate algorithmic behaviours such as recursion and composition when presented with appropriate prompts.
- Evaluation of compositional generalization must consider not only architectures and training data, but also the expressiveness of prompts and the presence of explicit decompositional structure.

This challenges the view that poor SCAN performance of standard sequence-to-sequence models necessarily indicates a fundamental lack of compositional capacity; instead, it suggests that at least some of this capacity can emerge from prompt-elicited behaviours in sufficiently large pretrained models.

### 4.2 Prompting as a form of program specification

Least-to-most prompting effectively treats the prompt as a high-level program specification:

- Decomposition prompts define how to break tasks into subroutines.
- Solution prompts define how to apply subroutines and reuse intermediate results.

This framing connects prompting to ideas in program synthesis and neuro-symbolic reasoning, where instructions are provided in a metalanguage and executed by a learned engine. The success of least-to-most prompting implies that LLMs can act as flexible interpreters for simple, human-written algorithms given sufficient guidance in natural language and examples.

### 4.3 Revisiting the role of intermediate reasoning in LLMs

Chain-of-thought prompting has popularised the idea of eliciting explicit rationales. Least-to-most prompting extends this by emphasising not only *length* of reasoning chains but also *structure* of reasoning via explicit decomposition into subproblems. The empirical results indicate that:

- Long monolithic reasoning chains are not always the most effective way to organise reasoning.
- Explicitly structuring reasoning as a sequence of subproblems with reuse can significantly improve performance on tasks that require many steps.

This suggests that research on intermediate reasoning should focus not only on eliciting explanations, but also on designing abstractions and interfaces that allow models to manipulate and reuse intermediate results in a modular way.

### 4.4 Implications for benchmark design and evaluation

The ease with which least-to-most prompting solves SCAN raises questions about the continued diagnostic value of certain benchmarks once prompt-based strategies are allowed. Benchmarks originally designed to stress compositionality under end-to-end training may be less discriminative when powerful pretrained models and elaborate prompt engineering are available. Future benchmarks may need to:

- explicitly constrain prompt size or structure,
- require out-of-distribution decompositions not easily captured by fixed exemplars,
- test robustness to varied decompositions or paraphrased instructions.

Similarly, the dependence of performance on decomposition quality suggests that future evaluations should measure not only final accuracy, but also the quality and diversity of model-generated decompositions.

## 5 Potential Research Directions

### 5.1 Automatic learning of decompositions

Given that decomposition quality is the dominant bottleneck, a natural research direction is to design mechanisms for *learning* or *searching* for decompositions rather than hand-crafting them. Possible approaches include:

- training smaller models to propose candidate decompositions that are then executed by a large model,
- using reinforcement learning or bandit algorithms to explore and optimise decomposition strategies based on downstream accuracy,
- framing decomposition as a structured prediction problem with supervision from human-written solutions or automatically generated synthetic data.

Such methods could transform least-to-most prompting from a manually engineered protocol into a more scalable, data-driven framework.

### 5.2 Interactive and bidirectional prompting schemes

The conclusion of the paper notes that prompting is essentially unidirectional. A promising line of work is to embed least-to-most prompting into interactive, bidirectional protocols in which:

- the model can ask clarification questions about ambiguous problem statements,
- human or automated feedback can correct incorrect decompositions or intermediate answers,
- multiple decomposition hypotheses can be explored in parallel and pruned using verification signals.

This could reduce the brittleness associated with a single decomposition path and align the prompting process more closely with human teaching practices.

### 5.3 Integration with training-time objectives

Least-to-most prompting is currently used as a purely inference-time strategy. Future work could integrate its core ideas into training procedures, for example by:

- training models to jointly predict decompositions and final answers, with explicit supervision on subproblems,
- using curriculum learning where models are progressively exposed to least-to-most-like tasks, starting from simpler decompositions and moving to more complex ones,
- incorporating loss functions that encourage consistency between solutions obtained via different decompositions of the same problem.

Such training-time integration may reduce the need for extremely large base models and could encourage models to internalise decompositional reasoning patterns.

### 5.4 Cross-domain and cross-task generality of decomposition schemas

Another direction is to investigate whether certain decomposition schemas can transfer across domains. For example:

- can decomposition prompts designed for arithmetic reasoning be adapted to logical reasoning or commonsense QA with minimal modifications,
- can generic schemas such as "identify entities, then compute relations between entities" be instantiated in different tasks using only task-specific lexical substitutions.

Systematic experiments on cross-domain generality would clarify which aspects of least-to-most prompting are domain-specific and which are reusable abstractions.

### 5.5 Robustness-enhancing variants of least-to-most prompting

To mitigate error propagation and prompt-length issues, several variants could be explored:

- employing self-consistency across decompositions instead of only across full reasoning chains,
- introducing verification steps that re-evaluate or re-derive subproblem answers from alternative decompositions,
- using compressed representations of intermediate results, such as structured key-value stores, rather than raw text concatenation, to reduce noise in the prompt.

These modifications could enhance robustness, especially in long chains of reasoning where compounding errors are most problematic.

## 6 Conclusion

The paper provides a clear and well-executed methodological contribution by introducing least-to-most prompting as a simple yet powerful strategy for eliciting complex reasoning from large language models. Through carefully designed experiments on symbolic manipulation, compositional generalization, and math reasoning, the study demonstrates that explicit problem decomposition and reuse of intermediate solutions can substantially improve easy-to-hard generalization over strong baselines, including chain-of-thought prompting.

At the same time, the work highlights a number of important limitations. The approach is heavily dependent on manually engineered prompts and is evaluated primarily on stylised benchmarks. Decomposition quality emerges as a central bottleneck, and the protocol introduces additional computational and robustness challenges due to longer prompts and sequential dependence.

The most promising research directions suggested by this analysis involve automating decomposition discovery, embedding least-to-most ideas into interactive and training-time frameworks, and extending evaluation to more diverse and realistic domains. Overall, the paper provides both an effective practical technique and a useful conceptual lens for thinking about how prompting strategies can shape and expose the latent reasoning capabilities of large language models.